Tasques 2 i 3 – Polsador

SOS en Morse i polsador com a interruptor

Pau Martínez Cuadra i Enric Vinyes Huguet
Robòtica · 1r Batxillerat · 20/10/2026

1. Objectiu

En aquestes dues pràctiques comencem a fer servir un polsador, o sigui que ara l'Arduino no només dona ordres al led, sinó que també llegeix si estem apretant el botó.

- **Tasca 2:** quan apretem el polsador, el led ha de fer SOS en Morse (tres curts, tres llargs i tres curts). Quan no l'apretem, ha d'estar apagat.
- **Tasca 3:** fer servir el polsador de 4 potes com un interruptor. Si l'apretes un cop el led s'encén i es queda encès, i si l'apretes una altra vegada s'apaga i es queda apagat.

Les dues tasques les vam fer amb el mateix muntatge, només canvia el programa.

2. Material

- Arduino UNO i cable USB
- Protoboard
- Un led (el vermell del port 10)
- Un polsador de 4 potes
- Una resistència per al polsador
- Cables
- Ordinador amb l'Arduino IDE

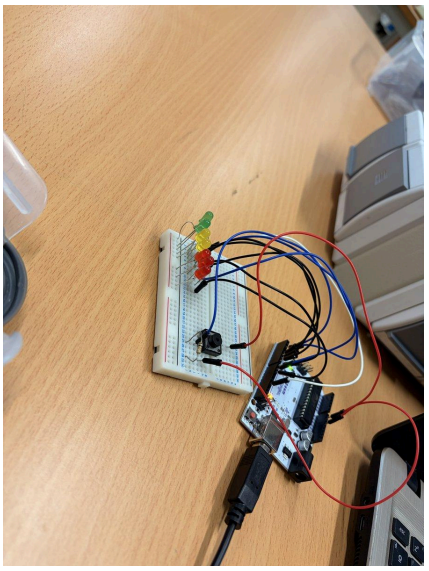
3. Muntatge

Vam aprofitar el muntatge de la Tasca 1.1, que encara tenia els 7 leds posats. Per aquestes tasques només fem servir el de baix de tot, el vermell que va al port 10. Al costat vam posar el polsador i la resistència.

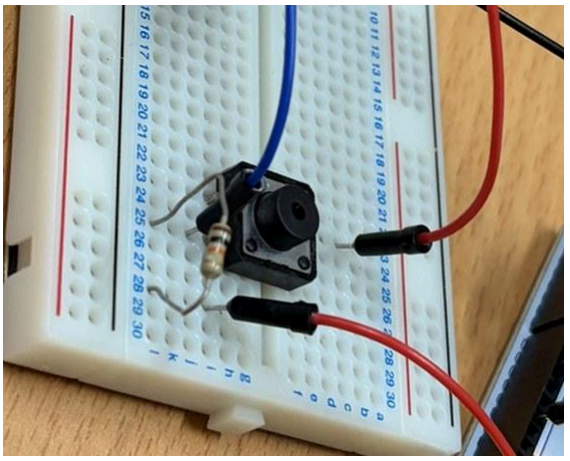
El polsador té 4 potes, però en realitat són dues parelles que ja estan unides per dins. Quan l'apretes, les dues parelles es connecten i hi passa el corrent. Les connexions ens van quedar així:

Què	Va connectat a
Led, pota llarga (+)	Port 10
Led, pota curta (−)	GND
Polsador, una pota	5V
Polsador, l'altra pota	Port 2 i la resistència
Resistència, l'altra punta	GND

La resistència és el que s'anomena pull-down. Serveix perquè quan no apretes el botó, el port 2 estigui connectat a GND i llegeixi LOW segur. Si no la poséssim, el port quedaria a l'aire i podria llegir coses rares. Quan apretes, li arriben els 5V i llegeix HIGH.



El muntatge sencer



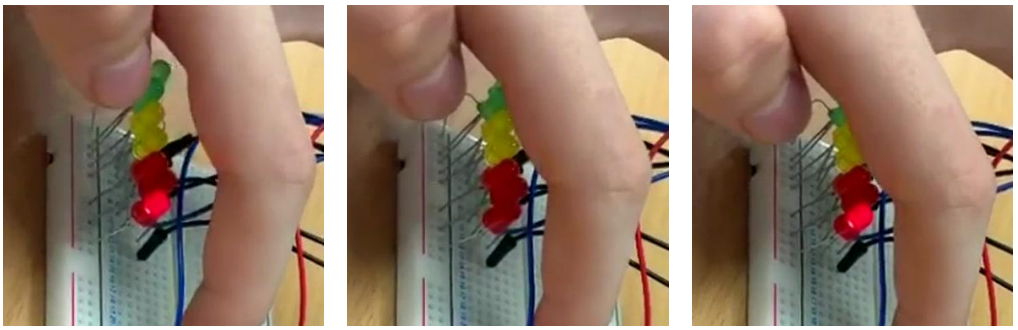
El polsador amb la resistència

4. Tasca 2: SOS amb el polsador

En Morse, la S són tres punts (· · ·) i la O tres ratlles (— — —). Els temps que vam fer servir són aquests:

Què	Variable	Temps
Punt (led encès)	punt	200 ms
Ratlla (led encès)	ratlla	600 ms
Apagat entre senyals	punt	200 ms
Pausa entre lletres	pausaLletra	400 ms més
Pausa al final de l'SOS	pausaMissatge	1200 ms més

Després de cada senyal sempre hi ha 200 ms apagat, i al final de cada lletra s'hi sumen els 400 de pausaLletra, o sigui que entre lletra i lletra el led està 600 ms apagat. La ratlla dura tres vegades el punt i la pausa entre lletres també, que és com va el Morse de veritat.



Captures del vídeo de la Tasca 2: un punt de la S (led encès), la pausa entre lletres (apagat) i una ratlla de la O (encès).

Et vam ensenyar dos codis que fan el mateix. Els posem tots dos.

4.1. Primer codi: amb for

```
int pinLED=10;
int boto=2;
int punt=200;
int ratlla=600;
int pausaLletra=400;
int pausaMissatge=1200;
int n=0;
int repos;

void setup() {
  pinMode(pinLED, OUTPUT);
  pinMode(boto, INPUT_PULLUP);
  delay(50);
  repos=digitalRead(boto);
}

void loop() {
  if (digitalRead(boto)!=repos) {
    for (n=0; n<3; n++) {
      digitalWrite(pinLED, HIGH);
      delay(punt);
      digitalWrite(pinLED, LOW);
      delay(punt);
    }
    delay(pausaLletra);
    for (n=0; n<3; n++) {
      digitalWrite(pinLED, HIGH);
      delay(ratlla);
      digitalWrite(pinLED, LOW);
      delay(punt);
    }
    delay(pausaLletra);
    for (n=0; n<3; n++) {
      digitalWrite(pinLED, HIGH);
      delay(punt);
      digitalWrite(pinLED, LOW);
      delay(punt);
    }
    delay(pausaMissatge);
  }
}
```

Al principi posem les variables: el pin del led, el del botó i els temps. La n és el comptador dels for, i repos és on guardem com està el botó quan no l'apretes.

Al setup posem el led com a OUTPUT i el botó com a INPUT_PULLUP. Esperem 50 ms i llegim el botó, i aquest valor el guardem a repos.

Això ho vam haver de fer perquè la primera versió no parava mai. Miràvem si el botó estava a LOW, però com que el nostre polsador té la resistència cap a GND, el port ja estava a LOW sense apretar res, i l'Arduino es pensava que l'estàvem apretant tota l'estona. Guardant la lectura a repos, al loop només fem l'SOS quan el botó està diferent de com està en repòs, és a dir, quan l'apretem.

El `for (n=0; n<3; n++)` és el que més ens va costar d'entendre. Vol dir: la n comença a 0, mentre la n sigui més petita que 3 es repeteix el que hi ha a dins, i a cada volta la n suma 1. Així fa 3 voltes (n=0, n=1 i n=2), que són els 3 punts o les 3 ratlles d'una lletra. Hi ha tres for: el de la primera S, el de la O (amb ratlla en lloc de punt) i el de la segona S.

4.2. Segon codi: sense for

Com que el for i tantes variables ens liaven, vam fer una altra versió més directa. És la que ens quedem, perquè l'entenem millor:

```
int pinLED=10;
int polsador=2;
int punt=200;
int ratlla=600;
int pausaLletra=400;
int pausaMissatge=1200;

void setup() {
  pinMode(pinLED, OUTPUT);
  pinMode(polsador, INPUT);
}

void loop() {
  if (digitalRead(polsador)==HIGH) {
    digitalWrite(pinLED, HIGH);
    delay(punt);
    digitalWrite(pinLED, LOW);
    delay(punt);
    digitalWrite(pinLED, HIGH);
    delay(punt);
    digitalWrite(pinLED, LOW);
    delay(punt);
    digitalWrite(pinLED, HIGH);
    delay(punt);
    digitalWrite(pinLED, LOW);
    delay(pausaLletra);
    digitalWrite(pinLED, HIGH);
    delay(ratlla);
    digitalWrite(pinLED, LOW);
    delay(punt);
    digitalWrite(pinLED, HIGH);
    delay(ratlla);
    digitalWrite(pinLED, LOW);
    delay(punt);
    digitalWrite(pinLED, HIGH);
    delay(ratlla);
    digitalWrite(pinLED, LOW);
    delay(punt);
    delay(pausaLletra);
    digitalWrite(pinLED, HIGH);
    delay(punt);
    digitalWrite(pinLED, LOW);
    delay(punt);
    digitalWrite(pinLED, HIGH);
    delay(punt);
    digitalWrite(pinLED, LOW);
    delay(punt);
    digitalWrite(pinLED, HIGH);
    delay(punt);
    digitalWrite(pinLED, LOW);
    delay(punt);
    delay(pausaMissatge);
  }
}
```

Aquí el botó és INPUT normal, i al loop només hi ha un `if (digitalRead(polsador)==HIGH)`. Com que tenim la resistència pull-down, quan no apretem el port llegeix LOW i no passa res, i quan apretem llegeix HIGH i entra a l'if.

A dins hi ha l'SOS escrit sencer: 9 vegades encendre, esperar, apagar i esperar. Les tres primeres amb punt (S), les tres següents amb ratlla (O) i les tres últimes amb punt una altra vegada (S). Entre lletra i lletra hi ha el delay(pausaLletra) i al final el delay(pausaMissatge).

Ja no ens calen ni la n ni repos. Els temps sí que els hem deixat en variables, perquè surten moltes vegades i així, si volguéssim fer-lo més ràpid, només hauríem de canviar un número.

4.3. Comparació i funcionament

	Amb for	**Sense for**
Llargada	Més curt	Més llarg (es repeteix el mateix 9 vegades)
Variables	6 de temps i pins, més n i repos	Només les de temps i pins
Botó	INPUT_PULLUP i compara amb repos	INPUT i mira si és HIGH
Per nosaltres	Costa més d'entendre	S'entén mirant-lo de dalt a baix

Els dos fan el mateix. Una S dura 3 × (200 + 200) = 1200 ms més la pausa de 400, o sigui

1,6 s. La O dura 3 × (600 + 200) = 2400 ms més 400, que són 2,8 s. L'última S són 1200 ms més la pausa final de 1200, 2,4 s. En total, un SOS dura 1,6 + 2,8 + 2,4 = 6,8 segons.

Una cosa que s'ha de saber és que l'Arduino només mira el botó a l'inici del loop. Si el deixes anar a meitat de l'SOS, l'acaba igualment i després para. I si el mantens apretat, va repetint l'SOS.

Vídeo de la Tasca 2: **veure el vídeo**

5. Tasca 3: el polsador com a interruptor

Aquí el led s'ha de quedar com està quan deixes anar el botó, i això és el que la fa diferent de la Tasca 2. Aquest és el codi:

```
int pinLED=10;
int polsador=2;

void setup() {
  pinMode(pinLED, OUTPUT);
  pinMode(polsador, INPUT);
}

void loop() {
  if (digitalRead(polsador)==HIGH) {
    if (digitalRead(pinLED)==LOW) {
      digitalWrite(pinLED, HIGH);
    } else {
      digitalWrite(pinLED, LOW);
    }
    delay(500);
  }
}
```

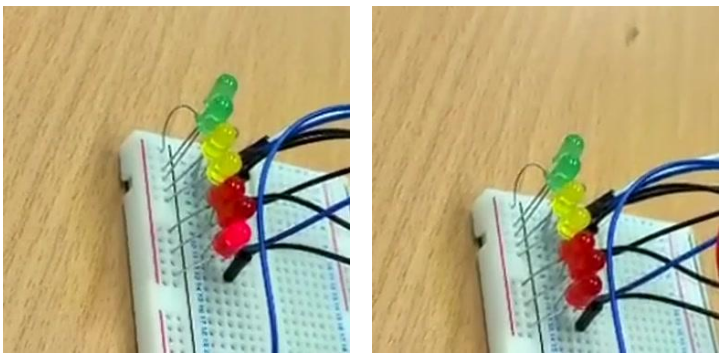
El setup és igual que el de l'SOS sense for: el led com a OUTPUT i el polsador com a INPUT.

Al loop, quan el polsador està a HIGH, mirem com està el led. Per saber-ho fem servir digitalRead amb el pin del led, perquè també es pot llegir un pin que és de sortida. Si el led està a LOW (apagat) l'encenem, i si no (o sigui, si està encès) l'apaguem. Així cada apretada canvia l'estat del led.

El delay(500) del final és important. El loop va rapidíssim i, mentre tens el dit al botó, passaria moltes vegades per l'if. El led s'encendria i s'apagaria molt ràpid i no sabries com quedaria. Amb el delay, després de canviar el led l'Arduino espera mig segon abans de tornar a mirar el botó, i dona temps a deixar-lo anar.

L'únic problema és que si mantens el botó apretat més de mig segon, torna a canviar, i el led fa pampallugues cada 0,5 s fins que el deixes anar. Si l'apretes normal, funciona bé.

Vídeo de la Tasca 3: **veure el vídeo**



Captures del vídeo de la Tasca 3: el led encès després d'apretar, i apagat quan l'apretem un altre cop.

6. Problemes

- El més gros va ser que el nostre primer SOS no parava mai. El codi feia servir INPUT_PULLUP i mirava si el botó estava a LOW, però amb la resistència cap a GND el port ja estava a LOW sense apretar. Ho vam arreglar primer guardant el repòs a repos, i després amb la versió que mira si està a HIGH.
- El for i tantes variables no els enteníem del tot. Per això vam fer la segona versió escrivint l'SOS sencer amb un if, encara que quedi més llarg.
- A la Tasca 3, la primera versió que vam tenir feia servir una variable per guardar l'estat i un while, i tampoc l'enteníem. La vam canviar per una de més directa que llegeix el mateix pin del led.

7. Conclusions

Les dues tasques ens han funcionat. Hem après a llegir un polsador amb digitalRead i que la resistència pull-down és la que fa que, sense apretar, es llegeixi LOW. Si no tens clar com està connectat el botó, el codi pot fer just el contrari del que vols, com ens va passar amb l'SOS que no parava.

També hem vist que un codi més curt no sempre és millor per nosaltres. El del for és més curt, però el sense for l'entenem de dalt a baix. I a la Tasca 3 hem après que es pot llegir el pin d'un led per saber si està encès, i que sense el delay el botó canviaria el led moltes vegades d'una sola apretada.